

Санкт-Петербургский государственный университет

Аверин Павел Владиславович

Выпускная квалификационная работа

Система для анализа мультиагентных алгоритмов определения позиций объектов

Уровень образования: бакалавриат

Направление *09.03.04 «Программная инженерия»*

Основная образовательная программа *СВ.5080.2022 «Программная инженерия»*

Научный руководитель:
д.ф.-м.н., проф., проф. каф. СП О. Н. Граничин

Рецензент:
старший инженер-программист, ООО «КНС ГРУПП» К.В. Кучеров

Санкт-Петербург
2026

Saint Petersburg State University

Pavel Averin

Bachelor's Thesis

Template for SPbU qualification works

Education level: bachelor

Speciality *09.03.04 "Software Engineering"*

Programme *CB.5080.2022 "Software Engineering"*

Scientific supervisor:
Sc.D, prof. O. N. Granichin

Reviewer:
Senior Software Engineer at "KNS Group" K. V. Kuchеров

Saint Petersburg
2026

Оглавление

Введение	4
1. Постановка задачи	6
2. Сбор требований	7
3. Обзор	9
3.1. Обзор существующих решений	9
4. Архитектура	12
4.1. Архитектура системы	12
4.2. Пользовательский сценарий	13
5. Особенности реализации	16
5.1. Используемый стек технологий	16
5.2. Симуляция	16
5.3. Реализованные алгоритмы	16
5.3.1. Алгоритм на основе SPSA	16
5.3.2. Ускоренный алгоритм на основе SPSA + Консенсус	17
Заключение	19
Список литературы	21

Введение

В современном мире информационные технологии и вычислительные системы стремительно развиваются. Вычислительная мощность устройств увеличивается с каждым годом, а их размеры уменьшаются.

В рамках данной работы рассматривается задача определения позиций (трекинг) объектов по расстояниям распределенных сенсоров в условиях помех. На фоне технологического прогресса решение такой задачи существенно усложнилось, поскольку объекты наблюдения стали более сложными, быстрыми и непредсказуемыми.

Традиционным подходом к решению данной задачи является централизованное решение, при котором выделяется единый вычислительный узел, собирающий данные с сенсоров, выполняющий необходимые вычисления и формирующий управляющее воздействие. У данного подхода можно выделить три слабых места:

- время на передачу данных от сенсоров до вычислителя;
- обработка данных и вычисления;
- время на передачу управляющего воздействия от вычислителя обратно к сенсорам.

Современные технологии позволяют перейти от этого метода к более новому и эффективному мультиагентному подходу, который устраняет недостатки централизованной архитектуры. В его рамках сенсоры сами выступают в роли вычислителей: они могут самостоятельно определять траекторию объектов, используя расстояния до целей, измеренные ими лично, а также, при необходимости, расстояния, полученные от соседних сенсоров.

К сожалению, на данный момент не существует единой системы с набором уже реализованных мультиагентных подходов, которая позволяла бы быстро и эффективно протестировать новый алгоритм и сравнить его с существующими аналогами в условиях возмущений и помех

не обладающих традиционным статистическим предположениями. Это существенно замедляет проведение исследований, так как исследователям приходится тратить время на самостоятельную реализацию других алгоритмов при отсутствии открытого исходного кода, а также на подготовку симуляций и экспериментов.

1. Постановка задачи

Целью данной дипломной работы является разработать систему для анализа мультиагентных алгоритмов для определения позиций объектов в условиях возмущений и помех не обладающих традиционным статистическим предположениями. Для достижения этой цели были сформулированы следующие задачи.

1. Провести обзор существующих систем для анализа мультиагентных алгоритмов.
2. Сформулировать требования к разрабатываемой системе.
3. Разработать архитектуру системы на основе требований.
4. Реализовать систему.
5. Интегрировать в систему существующие мультиагентные алгоритмы в качестве эталонов для сравнения и провести их валидацию.

2. Сбор требований

В результате взаимодействия с членами научной группы, занимающейся разработкой рассматриваемых алгоритмов, были сформулированы требования к разрабатываемой системе. Эти требования отражают как особенности предметной области, так и ограничения существующих инструментов.

1. **Поддержка пользовательских аккаунтов и загрузки алгоритмов.** Система должна обеспечивать возможность создания пользовательских аккаунтов, а также предоставлять интерфейс для загрузки и интеграции пользовательских алгоритмов. Это необходимо для проведения сравнительных исследований различных подходов в единой среде.
2. **Гибкая настройка параметров симуляции.** Должна быть предусмотрена возможность задания параметров сценария, включая количество целей и сенсоров, продолжительность моделирования, характер движения целей, а также параметры возмущений и помех. Это позволяет исследовать устойчивость алгоритмов в различных условиях.
3. **Визуализация результатов моделирования.** Система должна предоставлять средства для наглядного отображения результатов, включая траектории движения целей и эволюцию средне-квадратической ошибки. Визуализация является важным инструментом анализа качества алгоритмов.
4. **Поддержка пакетного запуска экспериментов.** Необходима возможность автоматизированного запуска серии сценариев с последующей агрегацией результатов. Это существенно упрощает проведение масштабных экспериментальных исследований.
5. **Сравнение результатов при различных параметрах.** Система должна обеспечивать средства для сравнения результатов работы алгоритмов при различных настройках симуляции, что поз-

воляет выявлять зависимости качества работы алгоритмов от параметров среды.

Сформулированные требования отражают необходимость в специализированной системе, ориентированной на проведение воспроизводимых и масштабируемых экспериментов с мультиагентными алгоритмами трекинга в условиях возмущений и помех.

3. Обзор

В данном разделе нужно описать всё, что необходимо для понимания Вашей работы и что придумали не Вы. В дальнейших разделах нельзя прерывать повествование, например, для рассказа о деталях используемой технологии или архитектуре старой системы, потому что читателю будет трудно отличить Ваш вклад от не Вашего.

Любой обзор пишется с какой-то целью (обосновать актуальность, найти и описать интересные решения, сравнить и выбрать технологии) и по какой-то методике поиска материала (например, поиск N релевантных статей на таких-то сервисах). Не будет лишним это всё явно описать.

3.1. Обзор существующих решений

В настоящее время существует ряд программных средств и библиотек, предназначенных для моделирования мультиагентных систем и трекинга объектов. Однако большинство из них либо ориентированы на централизованные методы обработки, либо не учитывают специфику рассматриваемой задачи — анализ мультиагентных алгоритмов в условиях возмущений и помех, не обладающих стандартными статистическими предположениями.

MATLAB Sensor Fusion and Tracking Toolbox предоставляет широкий набор инструментов для моделирования сенсоров, трекинга объектов и оценки качества алгоритмов. К его преимуществам относятся развитая экосистема, наличие готовых реализаций алгоритмов и удобные средства визуализации. Однако данный инструмент в первую очередь ориентирован на централизованные подходы к обработке данных. Кроме того, система является закрытой и требует дорогостоящей лицензии, а интеграция пользовательских мультиагентных алгоритмов затруднена.

Repast (Agent-Based Modeling Toolkit) представляет собой универсальную платформу для моделирования мультиагентных систем. К её преимуществам можно отнести гибкость и возможность описания

широкого класса агентных моделей. Тем не менее, данная платформа в основном применяется в социально-экономических и биологических задачах и не содержит встроенных средств для моделирования сенсоров и алгоритмов трекинга. Также стоит отметить, что система в настоящее время активно не развивается и ограниченно поддерживается.

SIMLAN и симуляторы на базе **Gazebo** широко используются в задачах робототехники для моделирования поведения роботов и тестирования алгоритмов навигации. Их основным преимуществом является высокая степень реалистичности физической симуляции и развитые средства интеграции с робототехническими платформами. Однако такие системы ориентированы на детальное моделирование физической среды и являются избыточными и сложными в настройке для задач абстрактного анализа алгоритмов трекинга. Кроме того, они не предоставляют удобных средств для массового сравнительного анализа алгоритмов.

Для наглядного сравнения рассмотренных решений приведём таблицу (табл. 1).

Таблица 1: Сравнение существующих решений

Характеристика	MATLAB Toolbox	Repast	Gazebo / SIMLAN
Поддержка мультиагентных алгоритмов	±	+	±
Встроенные модели сенсоров	+	-	+
Инструменты трекинга	+	-	±
Простота интеграции своих алгоритмов	-	+	-
Пакетные эксперименты и сравнение	±	-	-
Визуализация результатов	+	±	+
Ориентация на распределённые алгоритмы	-	±	±
Нестандартные статистические условия	-	-	-

Вывод. Проведённый анализ показал, что существующие решения либо ориентированы на централизованные методы обработки данных, либо не предоставляют необходимых инструментов для моделирования сенсорных систем и трекинга, либо избыточны для рассматриваемой задачи. Кроме того, ни одна из систем не поддерживает в полной мере проведение экспериментов в условиях нестандартных статистических предположений.

Таким образом, существует необходимость в разработке специа-

лизированной системы, позволяющей интегрировать пользовательские мультиагентные алгоритмы, гибко задавать параметры симуляции и проводить их сравнительный анализ в условиях возмущений и помех произвольной природы.

4. Архитектура

4.1. Архитектура системы

На основе требований, сформулированных в разделе сбора требований, была разработана модульная архитектура системы, обеспечивающая гибкость, расширяемость и возможность проведения воспроизводимых экспериментов.

Общая структура системы представлена на рисунке 1.

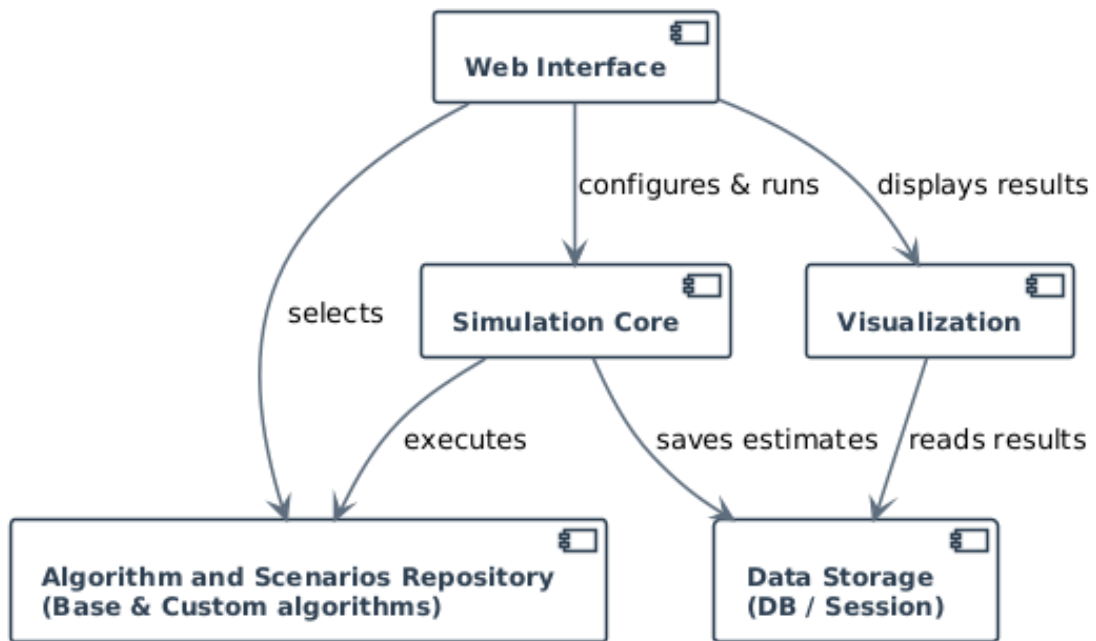


Рис. 1: Архитектура системы моделирования

Архитектура системы включает следующие основные компоненты:

- **Web Interface (веб-интерфейс).** Обеспечивает взаимодействие пользователя с системой. Через данный компонент осуществляется настройка параметров симуляции, выбор алгоритмов, запуск экспериментов и просмотр результатов.
- **Simulation Core (ядро моделирования).** Центральный компонент системы, отвечающий за выполнение симуляции, генерацию движения целей и обработку наблюдений с учётом шумов.

- **Algorithm Repository (репозиторий алгоритмов).** Содержит базовые и пользовательские алгоритмы и предоставляет унифицированный интерфейс для их выполнения.
- **Data Storage (хранилище данных).** Отвечает за сохранение результатов моделирования, параметров экспериментов и промежуточных вычислений.
- **Visualization (подсистема визуализации).** Обеспечивает отображение результатов в виде графиков траекторий и кривых ошибок.

Взаимодействие компонентов (рис. 1) организовано следующим образом: пользователь через веб-интерфейс задаёт параметры симуляции и выбирает алгоритмы, после чего управление передаётся в ядро моделирования. Ядро обращается к репозиторию алгоритмов, сохраняет результаты в хранилище и передаёт их в подсистему визуализации.

Таким образом, архитектура удовлетворяет требованиям: поддержки пользовательских алгоритмов, гибкой настройки, визуализации и пакетной обработки экспериментов.

4.2. Пользовательский сценарий

Типовой пользовательский сценарий работы с системой представлен на рисунке 2.

Сценарий работы пользователя включает следующие этапы:

1. **Открытие страницы настройки.** Пользователь переходит в веб-интерфейс и получает доступ к форме конфигурации.
2. **Задание параметров симуляции.** Пользователь задаёт длительность моделирования, количество целей и сенсоров, а также параметры шумов.
3. **Выбор алгоритмов.** Из репозитория выбираются алгоритмы для сравнения.

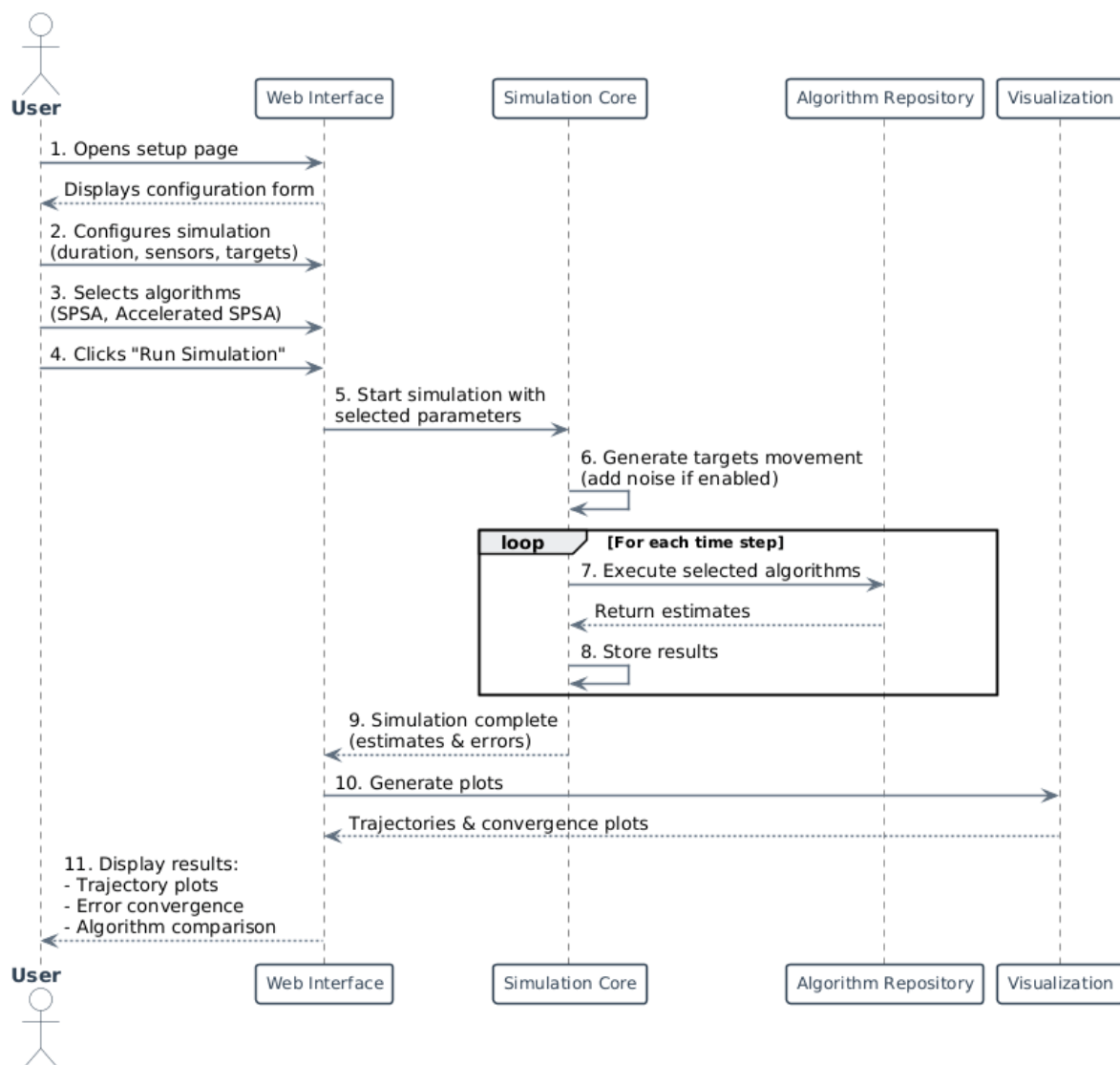


Рис. 2: Сценарий взаимодействия пользователя с системой

4. **Запуск симуляции.** Пользователь инициирует выполнение эксперимента.
5. **Выполнение моделирования.** Ядро моделирования выполняет расчёты, на каждом шаге времени вызывая выбранные алгоритмы.
6. **Получение результатов.** После завершения моделирования результаты передаются в систему визуализации.
7. **Анализ результатов.** Пользователь анализирует траектории, ошибки и сравнивает алгоритмы.

Дополнительно система поддерживает пакетный режим, позволяющий запускать серию экспериментов с последующим агрегированием результатов.

Таким образом, пользовательский сценарий (рис. 2) обеспечивает полный цикл работы с системой и соответствует сформулированным требованиям.

5. Особенности реализации

5.1. Используемый стек технологий

5.2. Симуляция

5.3. Реализованные алгоритмы

В рамках работы в систему интегрированы следующие мультиагентные алгоритмы определения позиций объектов.

5.3.1. Алгоритм на основе SPSA

$$\left\{ \begin{array}{l} \mathbf{x}_{2k}^i = \hat{\boldsymbol{\theta}}_{2k-2}^i + \beta_k^+ \Delta_k^i, \\ \mathbf{x}_{2k-1}^i = \hat{\boldsymbol{\theta}}_{2k-2}^i - \beta_k^- \Delta_k^i \\ \hat{\boldsymbol{\theta}}_{2k-1}^i = \hat{\boldsymbol{\theta}}_{2k-2}^i, \\ \hat{\boldsymbol{\theta}}_{2k}^i = \hat{\boldsymbol{\theta}}_{2k-1}^i - \alpha \left(\frac{y_{2k}^i - y_{2k-1}^i}{\beta_k} \mathbf{K}_k^i(\Delta_k^i) \right. \\ \left. + \gamma \sum_{j \in \mathcal{N}_{2k-1}^i} b_{2k-1}^{i,j} (\tilde{\theta}_{2k-1}^{i,j} - \hat{\theta}_{2k-1}^i) \right) \end{array} \right.$$

Пояснение основных шагов.

1. **Формирование текущей оценки.** Агент использует предыдущее состояние $\hat{\theta}_{2k-2}^i$ как базовую точку x_k^i .
2. **SPSA-возмущение.** Генерируется случайный вектор $\Delta_k^i \in \{\pm 1\}^d$ и формируются две точки: x_{2k}^i и x_{2k-1}^i .
3. **Оценка градиента.** Производится симметричная разностная аппроксимация:

$$\frac{y_{2k}^i - y_{2k-1}^i}{\beta_k} \Delta_k^i.$$

4. **Консенсусный шаг.** Добавляется вклад соседей через граф взаимодействия:

$$\sum_{j \in \mathcal{N}_i} b^{i,j} (\hat{\theta}^j - \hat{\theta}^i).$$

5. **Обновление параметров.** Выполняется градиентный шаг:

$$\hat{\theta}_{2k}^i = \hat{\theta}_{2k-1}^i - \alpha g_k^i.$$

5.3.2. Ускоренный алгоритм на основе SPSA + Консенсус

$$\left\{ \begin{array}{l} \bar{\mathbf{x}}_k = \frac{1}{\gamma_{k-1} + \alpha_k(\mu - \eta)} \left(\alpha_k \gamma_{k-1} \bar{\mathbf{z}}_{k-1} + \gamma_k \bar{\boldsymbol{\theta}}_{2k-1} \right), \\ \bar{\mathbf{x}}_{2k} = \bar{\mathbf{x}}_k + \beta_k^+ \bar{\boldsymbol{\Delta}}_k, \quad \bar{\mathbf{x}}_{2k-1} = \bar{\mathbf{x}}_k - \beta_k^- \bar{\boldsymbol{\Delta}}_k, \\ \bar{\mathbf{g}}_k = \left(\frac{\bar{\mathbf{y}}_{2k} - \bar{\mathbf{y}}_{2k-1}}{\beta_k} \otimes \mathbf{I}_d \right) \bar{\boldsymbol{\Delta}}_k + \omega \left(\bar{\mathcal{L}}_{2k-1} \otimes E_n \right) \bar{\mathbf{x}}_k, \\ \bar{\boldsymbol{\theta}}_{2k} = \bar{\mathbf{x}}_k - h \bar{\mathbf{g}}_k, \\ \bar{\boldsymbol{\theta}}_{2k+1} = \bar{\boldsymbol{\theta}}_{2k}, \\ \bar{\mathbf{z}}_k = \frac{(1 - \alpha_k) \gamma_{k-1} \bar{\mathbf{z}}_{k-1} + \alpha_k (\mu - \eta) \bar{\mathbf{x}}_k - \alpha_k \bar{\mathbf{g}}_k}{\gamma_k} \end{array} \right.$$

Пояснение основных шагов.

1. **Формирование точки зондирования $\tilde{x}_k$.** Это взвешенная комбинация предыдущего состояния θ и вспомогательной переменной z_k . Она играет роль “ускоренной” точки, аналогично методам Нестерова.
2. **Случайное возмущение (SPSA).** Генерируется случайный вектор Δ_k (обычно Бернулли ± 1). Затем вычисляются две точки:

$$x_{2k}, \quad x_{2k-1}$$

что позволяет оценить градиент без явных производных.

3. **Оценка градиента.** Градиент аппроксимируется как

$$g_k \approx \frac{f(x + \beta\Delta) - f(x - \beta\Delta)}{2\beta} \Delta,$$

с добавлением консенсусного члена ωLx_k , учитывающего взаимодействие агентов.

4. **Шаг спуска.** Выполняется обновление:

$$\theta_{2k} = \tilde{x}_k - hg_k,$$

где h — шаг обучения.

5. **Ускорение (переменная z_k).** Вводится вспомогательная последовательность z_k , которая аккумулирует информацию о предыдущих итерациях и обеспечивает ускоренную сходимость.

6. **Обновление параметров ускорения.** Параметры α_k и γ_k обновляются рекурсивно:

$$\gamma_k = (1 - \alpha_k)\gamma_{k-1} + \alpha_k(\mu - \eta),$$

что задаёт динамику ускорения.

Заключение

В ходе выполнения работы были получены следующие результаты.

- Проведён обзор существующих инструментов для моделирования и анализа мультиагентных систем, включая MATLAB Sensor Fusion and Tracking Toolbox, Repast (Agent-Based Modeling Toolkit) и SIMLAN, с точки зрения их применимости к задачам трекинга.
- Собраны и систематизированы требования от членов научной группы, разрабатывающих современные мультиагентные алгоритмы трекинга. Требования структурированы по ключевым группам: генерация сценариев (сенсоры, объекты, помехи), поддержка алгоритмов (базовые реализации и возможность интеграции пользовательских), а также средства сравнительного анализа (визуализация ошибок и траекторий).
- Спроектирована модульная архитектура системы, обеспечивающая разделение компонентов симуляции, выполнения алгоритмов и визуализации. Разработан пользовательский сценарий, в рамках которого исследователь задаёт параметры эксперимента, запускает вычислительный конвейер и получает наглядное сравнение эффективности алгоритмов.
- Реализована программная система в виде веб-приложения на основе фреймворка Django, обеспечивающая доступ к функциональности через удобный пользовательский интерфейс.
- Интегрированы эталонные мультиагентные алгоритмы трекинга: алгоритм на основе SPSSA, его ускоренная версия и распределённый фильтр Калмана, используемые в качестве базовой линии для сравнения новых алгоритмов.
- Результаты работы представлены на XXVII конференции молодых учёных «Навигация и управление движением», а также опубликованы в журнале IEEE Transactions on Automatic Control.

Исходный код разработанной системы: *[добавьте ссылку на репозиторий]*.

Список литературы